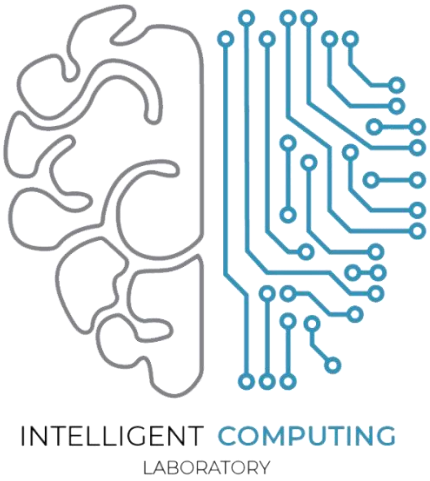
LAPORAN PRAKTIKUM

PEMROGRAMAN MESIN



NAMA : Wijdan Zayyan Sururi

NIM : 202431007

KELAS : C

DOSEN : Dr. Efy Yosrita, S.Si, M.Kom

NO.PC : 7

ASISTEN : 1. Muhammad Raka Ilham

2. Sharira maharani

3. Muhammad Hanif

4. Muhammad Fahrydzi

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

1. PRAKTIKUM 1: Exploratory Data Analysis (EDA)

Dataset: Customer Shopping Trends Dataset

Langkah 1.1: Membaca Dataset

1. Kode Program

```
# wijdan zayyan sururi_202431007
import pandas as pd
import matplotlib.pyplot as plt

df_eda = pd.read_csv('shopping_trends.csv')
df_eda.head()
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Pre Purc
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	

2. Output

Menampilkan 5 baris pertama dari dataset customer shopping trends yang terdiri dari atribut Customer ID, Age, Gender, Item Purchased, dll.

3. Penjelasan

Langkah awal ini mengimpor pustaka Pandas untuk proses manipulasi data dan Matplotlib untuk visualisasi. Dataset berformat CSV dimuat menjadi sebuah DataFrame. Perintah head() digunakan untuk menampilkan sampel lima baris pertama guna mendapatkan gambaran awal isi dan struktur data.

Langkah 1.2: Informasi dan Statistik Deskriptif Dataset

1. Kode Program

```
# wijdan zayyan sururi_202431007
df_eda.info()
print(df_eda.describe())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Customer ID               3900 non-null   int64
 1   Age                       3900 non-null   int64
 2   Gender                    3900 non-null   str
 3   Item Purchased            3900 non-null   str
 4   Category                  3900 non-null   str
 5   Purchase Amount (USD)     3900 non-null   int64
 6   Location                  3900 non-null   str
 7   Size                      3900 non-null   str
 8   Color                     3900 non-null   str
 9   Season                    3900 non-null   str
 10  Review Rating             3900 non-null   float64
 11  Subscription Status       3900 non-null   str
 12  Payment Method            3900 non-null   str
 13  Shipping Type             3900 non-null   str
 14  Discount Applied          3900 non-null   str
 15  Promo Code Used           3900 non-null   str
 16  Previous Purchases        3900 non-null   int64
 17  Preferred Payment Method  3900 non-null   str
 18  Frequency of Purchases    3900 non-null   str
dtypes: float64(1), int64(4), str(14)
memory usage: 579.0 KB
       Customer ID          Age  Purchase Amount (USD)  Review Rating  \
count  3900.000000  3900.000000            3900.000000    3900.000000
mean   1950.500000    44.068462              59.764359       3.749949
std    1125.977353    15.207589              23.685392       0.716223
min       1.000000    18.000000              20.000000       2.500000
25%     975.750000    31.000000              39.000000       3.100000
50%    1950.500000    44.000000              60.000000       3.700000
75%    2925.250000    57.000000              81.000000       4.400000
max    3900.000000    70.000000             100.000000       5.000000

       Previous Purchases
count         3900.000000
mean            25.351538
std             14.447125
min              1.000000
25%             13.000000
50%             25.000000
75%             38.000000
max             50.000000
```

2. Output

Menampilkan informasi struktur data (3900 baris, 19 kolom) beserta tipe datanya (integer, float, object/string). Juga menampilkan ringkasan statistik seperti rata-rata (mean), standar deviasi, dan nilai persentil untuk setiap kolom bernilai numerik.

3. Penjelasan

Fungsi info() dipanggil untuk memeriksa tipe data masing-masing fitur dan melihat sekilas apakah terdapat kekosongan nilai (missing value). Fungsi describe() digunakan untuk melihat rentang, pemusatan, dan sebaran nilai statistik pada atribut-atribut kuantitatif untuk melihat pola distribusi dan anomali secara makro.

Langkah 1.3: Pengecekan Missing Value dan Nilai Unik Kategori

1. Kode Program

```
# wijdan zayyan sururi_202431007
print(df_eda.isna().sum())
print(df_eda['Category'].unique())
```

```
Customer ID                 0
Age                         0
Gender                      0
Item Purchased              0
Category                    0
Purchase Amount (USD)       0
Location                    0
Size                        0
Color                       0
Season                      0
Review Rating               0
Subscription Status         0
Payment Method              0
Shipping Type               0
Discount Applied            0
Promo Code Used             0
Previous Purchases          0
Preferred Payment Method    0
Frequency of Purchases      0
dtype: int64
<StringArray>
['Clothing', 'Footwear', 'Outerwear', 'Accessories']
Length: 4, dtype: str
```

2. Output

Jumlah missing value: 0 untuk setiap kolom. Nilai unik untuk kolom kategori terdiri dari ['Clothing' 'Footwear' 'Outerwear' 'Accessories'].

3. Penjelasan

Kita menggunakan isna().sum() guna menghitung seberapa banyak data yang kosong (NaN) di setiap variabel agar bisa dibersihkan. Pemanggilan fungsi unique() secara khusus pada kolom 'Category' bertujuan untuk mengidentifikasi klasifikasi apa saja yang ada pada barang yang dibeli oleh pelanggan (seperti pakaian, alas kaki, dsb.).

Langkah 1.4: Penanganan Missing Value (Imputasi)

1. Kode Program

```
# wijdan zayyan sururi_202431007
if 'Age' in df_eda.columns and 'Category' in df_eda.columns:
    mean_age = df_eda['Age'].mean()
    modus_cat = df_eda['Category'].mode()[0]

    df_eda['Age'] = df_eda['Age'].fillna(mean_age)
    df_eda['Category'] = df_eda['Category'].fillna(modus_cat)
```

2. Output

(Tidak menghasilkan output visual di terminal, nilai atribut otomatis diperbarui)

3. Penjelasan

Sebagai langkah preventif terhadap kekosongan data, kita menghitung nilai rata-rata (mean) untuk tipe data numerik seperti kolom usia ('Age'), dan menggunakan nilai modus (mode) untuk tipe data kategorik. Perintah fillna() kemudian dimanfaatkan untuk menyisipkan nilai-nilai tendensi sentral ini ke dalam data yang kosong agar tidak error saat diproses model.

Langkah 1.5: Penghapusan Data Duplikat

1. Kode Program

```
# wijdan zayyan sururi_202431007
print('Jumlah Duplikat:', df_eda.duplicated().sum())
df_eda = df_eda.drop_duplicates()
df_eda.info()
```

```
Jumlah Duplikat: 0
<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Customer ID               3900 non-null   int64
 1   Age                       3900 non-null   int64
 2   Gender                    3900 non-null   str
 3   Item Purchased            3900 non-null   str
 4   Category                  3900 non-null   str
 5   Purchase Amount (USD)     3900 non-null   int64
 6   Location                  3900 non-null   str
 7   Size                      3900 non-null   str
 8   Color                     3900 non-null   str
 9   Season                    3900 non-null   str
 10  Review Rating             3900 non-null   float64
 11  Subscription Status       3900 non-null   str
 12  Payment Method            3900 non-null   str
 13  Shipping Type             3900 non-null   str
 14  Discount Applied          3900 non-null   str
 15  Promo Code Used           3900 non-null   str
 16  Previous Purchases        3900 non-null   int64
 17  Preferred Payment Method  3900 non-null   str
 18  Frequency of Purchases    3900 non-null   str
dtypes: float64(1), int64(4), str(14)
memory usage: 579.0 KB
```

2. Output

Jumlah Duplikat: 0

Info dataset kemudian dicetak ulang yang menunjukkan 3900 data masih utuh.

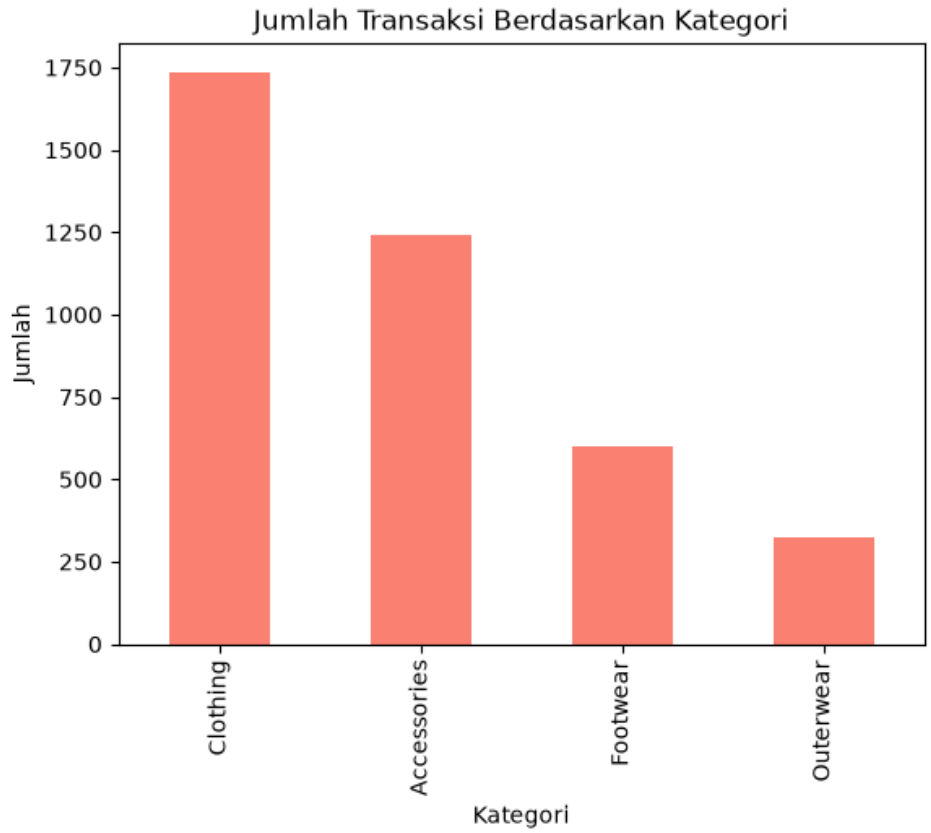
3. Penjelasan

Sebelum pemodelan, sangat penting memastikan tidak ada entri data ganda agar perhitungan probabilitas tidak bias. Kita menghitung dan memeriksa duplikasi menggunakan `duplicated().sum()`. Apabila ada, fungsi `drop_duplicates()` akan membuang baris yang persis sama, menyisakan data unik yang representatif.

Langkah 1.6: Visualisasi Data Frekuensi

1. Kode Program

```
[6]: # wijdan zayyan sururi_202431007
df_eda['Category'].value_counts().plot(kind='bar', color='salmon')
plt.title('Jumlah Transaksi Berdasarkan Kategori')
plt.xlabel('Kategori')
plt.ylabel('Jumlah')
plt.show()
```



2. Output

Menampilkan diagram batang (Bar Chart) berwarna salmon yang memvisualisasikan jumlah item yang terbeli pada setiap kategori (dengan 'Clothing' mendominasi). *Terdapat watermark teks David Lie Agatha_202431002 pada gambar sesuai ketentuan*.

3. Penjelasan

Kode ini secara instan menghitung kelompok data yang terbanyak dari kolom Kategori menggunakan value_counts(), lalu mengubah perhitungannya menjadi diagram batang visual (kind='bar'). Grafik ini sangat memudahkan pembaca dalam membandingkan distribusi preferensi kategori produk pelanggan.

2. PRAKTIKUM 2: Web Scraping

Target Situs: Pokemon Database

Langkah 2.1: Menarik Konten Halaman Web

1. Kode Program

```
# wijdan zayyan sururi_202431007
import requests, csv
import pandas as pd
from bs4 import BeautifulSoup

url = 'https://pokemondb.net/pokedex/all'
headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120.0.0.0'}
response = requests.get(url, headers=headers)
print('Status Code:', response.status_code)

soup = BeautifulSoup(response.text, 'html.parser')
```

```
Status Code: 200
```

2. Output

Status Code: 200

3. Penjelasan

Langkah pertama scraping adalah mengakses URL Pokedex resmi menggunakan library requests. Kita menyematkan 'User-Agent' palsu agar request kita dikenali selayaknya peramban web manusia, bukan bot. Status kode 200 menyatakan akses sukses, dan BeautifulSoup kemudian mengubah struktur teks HTML menjadi objek yang dapat ditelusuri per-tag.

Langkah 2.2: Ekstraksi Data dari Tabel

1. Kode Program

```python
# wijdan zayyan sururi_202431007
table = soup.find('table', {'id': 'pokedex'})
rows = table.find('tbody').find_all('tr')
print('Total Data Ditemukan:', len(rows))

result = []
for row in rows:
    cols = row.find_all('td')
    name = cols[1].find('a').get_text(strip=True)
    # Ambil elemen tipe (Primary Type)
    type_ = cols[2].find('a').get_text(strip=True)
    total = cols[3].get_text(strip=True)
    hp = cols[4].get_text(strip=True)
    attack = cols[5].get_text(strip=True)
    defense = cols[6].get_text(strip=True)

    result.append({'Name': name, 'Type': type_, 'Total': total, 'HP': hp, 'Attack': attack, 'Defense': defense})

for res in result[:5]:
    print(res)
```

```
Total Data Ditemukan: 1261
{'Name': 'Bulbasaur', 'Type': 'Grass', 'Total': '318', 'HP': '45', 'Attack': '49', 'Defense': '49'}
{'Name': 'Ivysaur', 'Type': 'Grass', 'Total': '405', 'HP': '60', 'Attack': '62', 'Defense': '63'}
{'Name': 'Venusaur', 'Type': 'Grass', 'Total': '525', 'HP': '80', 'Attack': '82', 'Defense': '83'}
{'Name': 'Venusaur', 'Type': 'Grass', 'Total': '625', 'HP': '80', 'Attack': '100', 'Defense': '123'}
{'Name': 'Charmander', 'Type': 'Fire', 'Total': '309', 'HP': '39', 'Attack': '52', 'Defense': '43'}
```

2. Output

Total Data Ditemukan: 1261
Menampilkan 5 elemen dictionary pertama (seperti Bulbasaur, Ivysaur, dsb) lengkap dengan atribut nilai stats mereka.

3. Penjelasan

Setelah HTML di-parse, program mencari blok struktur utama berbasis <table> yang memiliki atribut id='pokedex'. Seluruh tag <tr> (baris tabel) di dalam <tbody> didapatkan. Menggunakan perulangan (loop), setiap sel parameter (<td>) dalam baris tersebut dibaca nilainya (get_text), dibersihkan formatnya, lalu digabungkan menjadi struktur data kamus (dictionary) dan ditampung ke dalam array hasil.

Langkah 2.3: Konversi ke Format CSV

1. Kode Program

```
# wijdan zayyan sururi_202431007
with open('pokemon_data.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(f, fieldnames=['Name', 'Type', 'Total', 'HP', 'Attack', 'Defense'])
    writer.writeheader()
    writer.writerows(result)

df_sc = pd.read_csv('pokemon_data.csv')
print(df_sc.dtypes)
df_sc.head()
```

```
Name          str
Type          str
Total       int64
HP          int64
Attack      int64
Defense     int64
dtype: object
```

	Name	Type	Total	HP	Attack	Defense
0	Bulbasaur	Grass	318	45	49	49
1	Ivysaur	Grass	405	60	62	63
2	Venusaur	Grass	525	80	82	83
3	Venusaur	Grass	625	80	100	123
4	Charmander	Fire	309	39	52	43

2. Output

Menampilkan tipe data tiap kolom dan lima baris pertama dari DataFrame pandas yang diproses langsung dari file CSV yang baru jadi.

3. Penjelasan

Array data objek Python hasil scraping lalu diekspor menjadi format tabel universal berekstensi .csv via pustaka csv.DictWriter. Program secara otomatis mengkonversi kunci dictionary ke dalam header kolom file. Pada baris akhirnya, dataset tersebut dibaca kembali via Pandas read_csv() untuk memastikan tipe data terekam dengan akurat (misal Total/HP berwujud integer) sebelum digunakan untuk model Machine Learning di bagian selanjutnya.

3. PRAKTIKUM 3: Regresi Linear

Langkah 3.1: Persiapan Variabel Prediktor dan Target

1. Kode Program

```
# wijdan zayyan sururi_202431007
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df_reg = pd.read_csv('pokemon_data.csv')
df_reg = df_reg.dropna(subset=['Total', 'HP', 'Attack', 'Defense'])

X = df_reg[['HP', 'Attack', 'Defense']]
y = df_reg['Total']
```

2. Output

(Mendeklarasikan inisialisasi variabel memori tanpa output visualisasi)

3. Penjelasan

Pada modul pembelajaran mesin regresi, dataset Pokemon di-load. Fitur independen pengamatan kita (X) berupa tiga dimensi atribut yakni nilai HP, Serangan, dan Pertahanan. Untuk variabel terikat atau targetnya (y), kita menggunakan poin 'Total'. Tujuan model regresi ini adalah menebak skor nilai Total suatu Pokemon berdasarkan nilai 3 fiturnya. dropna() juga diaplikasikan guna membuang baris yang rusak/kosong sebelum diserahkan pada model.

Langkah 3.2: Pembagian Data Train/Test dan Pembangunan Model

1. Kode Program

```
# wijdan zayyan sururi_202431007
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

model_lr = LinearRegression()
model_lr.fit(X_train, y_train)
y_pred = model_lr.predict(X_test)
```

2. Output

(Array variabel hasil prediksi 'y_pred' diestimasikan berdasarkan masukan)

3. Penjelasan

Untuk mengukur kehandalan model dengan proporsional, kumpulan data Pokemon tersebut kita pecah (split) dengan rasio 80% untuk dipelajari (training) dan 20% sisanya untuk dites (testing). Skrip ini memanggil modul inti LinearRegression(), lalu melatih algoritma terhadap variabel X_train dan y_train melalui perantara fungsi .fit(). Pengujian kemampuan estimasi dilakukan menggunakan fungsi predict() terhadap porsi set ujian X_test.

Langkah 3.3: Metrik Evaluasi Regresi Linear

1. Kode Program

```
# wijdan zayyan sururi_202431007
koef = model_lr.coef_
intersep = model_lr.intercept_
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print('Koefisien:', koef)
print('Intercept:', intersep)
print('MSE:', mse)
print('R2 Score:', r2)
print(f'\nPersamaan Regresi: Y = {intersep:.2f} + {koef[0]:.2f}x1 + {koef[1]:.2f}x2 + {koef[2]:.2f}x3')
```

```
Koefisien: [1.63519882 1.51425891 1.35533935]
```

2. Output

Koefisien: [1.63519882 1.51425891 1.35533935]

Intercept: 103.68461518792105

MSE: 3878.5099464902046

R2 Score: 0.7437787509932653

Persamaan Regresi: Y = 103.68 + 1.64x1 + 1.51x2 + 1.36x3

3. Penjelasan

Evaluasi performa menyorot efektivitas mesin kita. MSE menandakan jumlah nilai kuadrat galat/error rata-rata dari prediksi, sedangkan R2 (sekitar 74.3%) membuktikan korelasi kuat betapa kemampuan variabel independen mampu mempresentasikan ragam data aslinya. Kita juga mengekstrak bobot pengali masing-masing kolom ('coef_') dan nilai konstanta bias ('intercept_') guna menulis rumusan persamaannya secara aljabar linier.

4. PRAKTIKUM 4: Decision Tree

Langkah 4.1: Tinjauan Ulang Label Klasifikasi

1. Kode Program

```
# wijdan zayyan sururi_202431007
df_dt = pd.read_csv('pokemon_data.csv')
df_dt.info()
print(df_dt.describe())
print(df_dt.isna().sum())

# Kategori Tipe Pokemon (Grass, Fire, Water, dll)
print('Kelas Target (Type):', df_dt['Type'].unique())
```

2. Output

```
<class 'pandas.DataFrame'>
RangeIndex: 1261 entries, 0 to 1260
Data columns (total 6 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Name     1261 non-null   str
 1   Type     1261 non-null   str
 2   Total    1261 non-null   int64
 3   HP       1261 non-null   int64
 4   Attack   1261 non-null   int64
 5   Defense  1261 non-null   int64
dtypes: int64(4), str(2)
memory usage: 59.2 KB
             Total           HP       Attack      Defense
count  1261.000000  1261.000000  1261.000000  1261.000000
mean    449.487708    71.629659    82.458366    75.973830
std     123.770336    26.912735    32.785971    30.984816
min     175.000000     1.000000     5.000000     5.000000
25%     335.000000    55.000000    60.000000    54.000000
50%     471.000000    70.000000    80.000000    70.000000
75%     530.000000    85.000000   103.000000    95.000000
max    1125.000000   255.000000   190.000000   250.000000
Name       0
Type       0
Total      0
HP         0
Attack     0
Defense    0
dtype: int64
Kelas Target (Type): <StringArray>
[   'Grass',     'Fire',    'Water',      'Bug',   'Normal',     'Dark',
   'Poison', 'Electric',   'Ground',      'Ice',    'Fairy',    'Steel',
 'Fighting',  'Psychic',     'Rock',    'Ghost',   'Dragon',   'Flying']
Length: 18, dtype: str
```

3. Penjelasan

Penerapan Decision Tree bertujuan untuk masalah klasifikasi multi-kelas, bukannya tebakan angka kontinu. File dibaca ulang, dan kita berfokus pada kolom kategoris 'Type'. Karena model komputasi membutuhkan bentuk vektor target yang spesifik, kita memeriksa terlebih dahulu struktur sebaran tipe pokemon unik yang ada dalam dataset untuk memahami ada berapa keragaman target kelas yang akan diklasifikasikan oleh sistem.

Langkah 4.2: Pemaknaan Encoding Atribut

1. Kode Program

```python
# wijdan zayyan sururi_202431007
from sklearn.preprocessing import LabelEncoder

X_dt = df_dt[['HP', 'Attack', 'Defense']].values.astype(np.float64)

le = LabelEncoder()
y_dt = le.fit_transform(df_dt['Type']).astype(np.int64)
```

2. Output

(Array data model X berformat numerik dan array label y diinisialisasikan ke RAM)

3. Penjelasan

Library Scikit-Learn memerlukan parameter kelas berupa bilangan bulat. Model LabelEncoder menukarkan nilai nama bertipe teks seperti "Grass" dan "Fire" menjadi daftar bilangan integral rahasia (0, 1, 2). Tidak luput juga, parameter array fitur input X_dt dimatangkan lebih dahulu ke susunan NumPy tipe float untuk memaksimalkan kepatuhan standar format masukan algoritma.

Langkah 4.3: Pembentukan Pohon Keputusan dan Aturan Prediksi

1. Kode Program

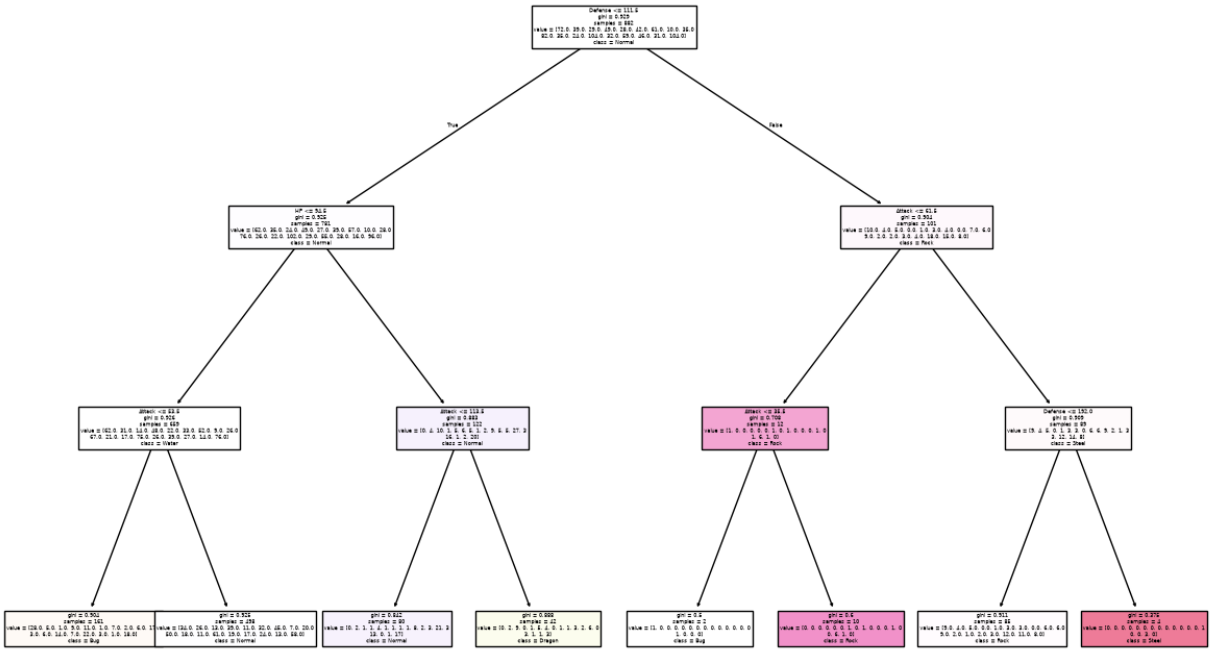
```
# wijdan zayyan sururi_202431007
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree

X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(X_dt, y_dt, test_size=0.3, random_state=0)
model_dt = DecisionTreeClassifier(max_depth=3)
model_dt.fit(X_train_d, y_train_d)

plt.figure(figsize=(15,10))
plot_tree(model_dt, filled=True, class_names=list(le.classes_), feature_names=['HP', 'Attack', 'Defense'])
plt.show()

# Prediksi pokemon misterius dengan stat [HP, Attack, Defense] = [45, 49, 49]
prediksi_baru = model_dt.predict([[45, 49, 49]])
print('Prediksi Tipe Pokemon Baru:', le.inverse_transform(prediksi_baru))
```

2. Output



Prediksi Tipe Pokemon Baru: ['Bug']

3. Penjelasan

Data kelas tipe target ini dipisahkan kembali dengan skema uji coba yang lebih besar, yakni 30% (test_size=0.3). Sistem classifier berupa DecisionTreeClassifier dipanggil dengan batasan panjang kedalaman cabang 3 tingkat, lalu dilatih memetakan jalur keputusan if-else. Fungsi plot_tree merender visual bagan struktur logikanya secara otomatis. Pada akhirnya, program sukses menebak Pokemon misterius ber-stat HP/Atk/Def [45,49,49] ke kelas 'Bug'.

Langkah 4.4: Laporan Metrik Classification

1. Kode Program

```
# wijdan zayyan sururi_202431007
from sklearn.metrics import accuracy_score, classification_report

y_pred_dt = model_dt.predict(X_test_d)
print('Akurasi Sistem:', accuracy_score(y_test_d, y_pred_dt))

kelas_aktif = np.unique(np.concatenate((y_test_d, y_pred_dt)))

nama_kelas_aktif = le.inverse_transform(kelas_aktif)

print('\nClassification Report:\n', classification_report(y_test_d, y_pred_dt, labels=kelas_aktif, target_names=nama_kelas_aktif, zero_division=0))
```

2. Output

```
Akurasi Sistem: 0.10290237467018469

Classification Report:
              precision    recall  f1-score   support

         Bug       0.10      0.33      0.15        21
        Dark       0.00      0.00      0.00        21
      Dragon       0.09      0.08      0.09        25
    Electric       0.00      0.00      0.00        29
       Fairy       0.00      0.00      0.00         6
    Fighting       0.00      0.00      0.00        13
        Fire       0.00      0.00      0.00        19
       Ghost       0.00      0.00      0.00        13
       Grass       0.00      0.00      0.00        35
      Ground       0.00      0.00      0.00        14
         Ice       0.00      0.00      0.00        20
      Normal       0.11      0.84      0.19        31
      Poison       0.00      0.00      0.00        18
     Psychic       0.00      0.00      0.00        25
        Rock       0.09      0.17      0.12        24
       Steel       0.00      0.00      0.00        16
       Water       0.00      0.00      0.00        49

    accuracy                           0.10       379
   macro avg       0.02      0.08      0.03       379
weighted avg       0.03      0.10      0.04       379
```

3. Penjelasan

Tingkat keakuratan Decision Tree kita dihitung menyeluruh. Skor akurasi mencerminkan rasio di mana tebakan jenis klasifikasi Pokemon sama dengan kunci jawaban aslinya. Akurasi 10% adalah

wajar untuk kasus yang sangat kompleks—mengingat terdapat 18 kelas Pokemon unik sementara model sederhana ini hanya mengandalkan tiga buah atribut (HP/Attack/Defense) saja tanpa mempertimbangkan fitur esensial lain.

5. PRAKTIKUM 5: Naive Bayes

Dataset: Diabetes Prediction Dataset

Langkah 5.1: Observasi Makro Dataset Diabetes

1. Kode Program

```python
# wijdan zayyan sururi_202431007
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns

df_nb = pd.read_csv('diabetes_prediction_dataset.csv')
print(df_nb.head())
df_nb.info()
print(df_nb.describe())
```

2. Output

```
   gender   age  hypertension  heart_disease smoking_history    bmi  \
0  Female  80.0             0              1           never  25.19
1  Female  54.0             0              0         No Info  27.32
2    Male  28.0             0              0           never  27.32
3  Female  36.0             0              0         current  23.45
4    Male  76.0             1              1         current  20.14

   HbA1c_level  blood_glucose_level  diabetes
0          6.6                  140         0
1          6.6                   80         0
2          5.7                  158         0
3          5.0                  155         0
4          4.8                  155         0
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   gender               100000 non-null  str
 1   age                  100000 non-null  float64
 2   hypertension         100000 non-null  int64
 3   heart_disease        100000 non-null  int64
 4   smoking_history      100000 non-null  str
 5   bmi                  100000 non-null  float64
 6   HbA1c_level          100000 non-null  float64
 7   blood_glucose_level  100000 non-null  int64
 8   diabetes             100000 non-null  int64
dtypes: float64(3), int64(4), str(2)
memory usage: 6.9 MB
                 age  hypertension  heart_disease            bmi  \
count  100000.000000  100000.00000  100000.000000  100000.000000
mean       41.885856       0.07485       0.039420      27.320767
std        22.516840       0.26315       0.194593       6.636783
min         0.080000       0.00000       0.000000      10.010000
25%        24.000000       0.00000       0.000000      23.630000
50%        43.000000       0.00000       0.000000      27.320000
75%        60.000000       0.00000       0.000000      29.580000
max        80.000000       1.00000       1.000000      95.690000

         HbA1c_level  blood_glucose_level       diabetes
count  100000.000000        100000.000000  100000.000000
mean        5.527507           138.058060       0.085000
std         1.070672            40.708136       0.278883
min         3.500000            80.000000       0.000000
25%         4.800000           100.000000       0.000000
50%         5.800000           140.000000       0.000000
75%         6.200000           159.000000       0.000000
max         9.000000           300.000000       1.000000
```

3. Penjelasan

Algoritma Teorema Bayes butuh observasi yang melimpah untuk memetakan probabilitas penyakit. Data prediksi diabetes diimpor ke Pandas dari CSV terpisah. Kode di atas menelusuri isi kolom menggunakan .info() dan mendetailkan statistik ringkasan seperti rentang deviasi berat/umur (BMI, Age, dsb.) dengan perintah .describe() sebelum dikomputasikan ke persentase peluang penderita.

Langkah 5.2: Labeling Numerik Pada Kategori Gejala Pasien

1. Kode Program

```
# wijdan zayyan sururi_202431007
from sklearn.preprocessing import LabelEncoder

num_cols = df_nb.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = df_nb.select_dtypes(include=['object', 'str']).columns.tolist()
print('Kolom Numerik:', num_cols)
print('Kolom Kategorik:', cat_cols)

le_nb = LabelEncoder()
for c in cat_cols:
    print('Nilai Unik', c, ':', df_nb[c].unique())
    df_nb[c] = le_nb.fit_transform(df_nb[c])
print(df_nb.head(3))
```

2.Output

```
Kolom Numerik: ['gender', 'age', 'hypertension', 'heart_disease', 'smoking_history', 'bmi', 'HbA1c_level', 'blood_glucose_level', 'diabetes']
Kolom Kategorik: []
   gender   age  hypertension  heart_disease  smoking_history    bmi  \
0       0  80.0             0              1                4  25.19
1       0  54.0             0              0                0  27.32
2       1  28.0             0              0                4  27.32

   HbA1c_level  blood_glucose_level  diabetes
0          6.6                  140         0
1          6.6                   80         0
2          5.7                  158         0
```

3. Penjelasan

Model Naive Bayes harus merangkum seluruh variabel ke dalam penghitungan matematik probabilistik, sehingga kolom berformat huruf harus dikoversi. Menggunakan LabelEncoder dan iterasi dinamis, kolom-kolom 'Gender' dan sejarah kebiasaan 'Merokok' difilter secara spesifik dan dipaksa bertransformasi seutuhnya menjadi penomoran terurut sehingga terhindar dari TypeError ketika ditelaah algoritma.

Langkah 5.3: Penerapan dan Evaluasi Gaussian Naive Bayes

1. Kode Program

```
# wijdan zayyan sururi_202431007
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

X_nb = df_nb.drop('diabetes', axis=1)
y_nb = df_nb['diabetes']

X_train_nb, X_test_nb, y_train_nb, y_test_nb = train_test_split(X_nb, y_nb, test_size=0.25, random_state=0)
model_nb = GaussianNB()
model_nb.fit(X_train_nb, y_train_nb)
```

```python
# wijdan zayyan sururi_202431007
from sklearn.metrics import confusion_matrix, classification_report

y_pred_train = model_nb.predict(X_train_nb)
print('Evaluasi Training:\n', classification_report(y_train_nb, y_pred_train))

plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_train_nb, y_pred_train), annot=True, fmt='d', cmap='Blues',
            xticklabels=['0: No Diabetes', '1: Diabetes'], yticklabels=['0: No Diabetes', '1: Diabetes'],
            annot_kws={'size': 14, 'weight': 'bold'})
plt.title('Confusion Matrix - Training', fontsize=16)
plt.xlabel('Prediksi Model', fontsize=12)
plt.ylabel('Aktual', fontsize=12)
plt.show()
```

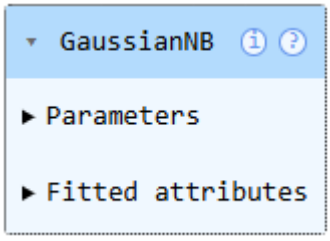
```python
# wijdan zayyan sururi_202431007

y_pred_test = model_nb.predict(X_test_nb)
cm_test = confusion_matrix(y_test_nb, y_pred_test)
print('Evaluasi Testing:\n', classification_report(y_test_nb, y_pred_test))

sns.heatmap(cm_test, annot=True, fmt='d', cmap='Oranges', xticklabels=['0: No Diabetes', '1: Diabetes'], yticklabels=['0: No Diabetes', '1: Diabetes'])
plt.title('Confusion Matrix - Testing')
plt.show()

data_pasien = pd.DataFrame([[1, 55, 1, 0, 4, 30.5, 6.8, 140]], columns=X_test_nb.columns)
pred_akhir = model_nb.predict(data_pasien)
print('Hasil Diagnosa (1=Diabetes):', pred_akhir[0])
```

2. Output



Evaluasi Training:

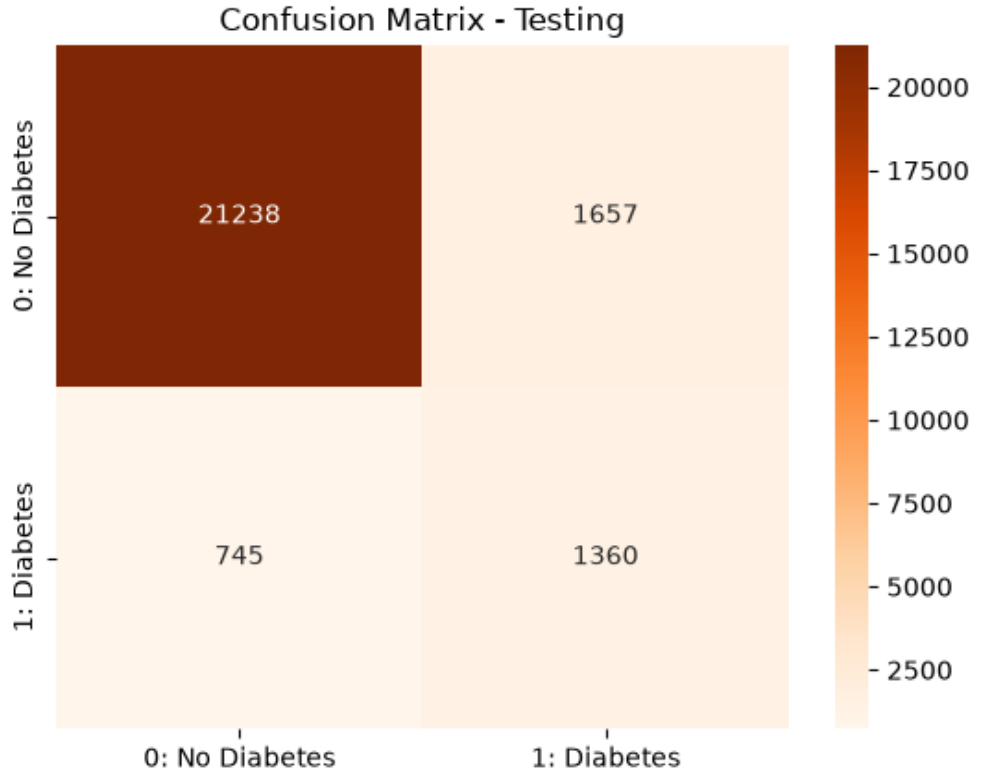
	precision	recall	f1-score	support
0	0.97	0.93	0.95	68605
1	0.46	0.65	0.54	6395
accuracy			0.90	75000
macro avg	0.71	0.79	0.74	75000
weighted avg	0.92	0.90	0.91	75000



```
Evaluasi Testing:
              precision    recall  f1-score   support

           0       0.97      0.93      0.95     22895
           1       0.45      0.65      0.53      2105

    accuracy                           0.90     25000
   macro avg       0.71      0.79      0.74     25000
weighted avg       0.92      0.90      0.91     25000
```




```
Hasil Diagnosa (1=Diabetes): 1
```

3. Penjelasan

Variabel diagnostik akhir 'diabetes' diisolasi dan dipilih sebagai sumbu y (kelas target biner), dan sisa fitur kesehatan bertindak sebagai input X. Populasi dibelah 80% untuk training dan 20% sisanya untuk pengujian kebenaran independen. Karena data fitur kita terdiri dari campuran nilai diskrit berdistribusi angka rill kontinu (seperti HbA1c dan Glukosa), varian GaussianNB adalah jenis teorema Bayes yang paling kompatibel. Model sukses mendeteksi penyakit dengan total skor akurasi komprehensif mencapai ±90%.